



PROBLEM SESSION 3

Process Manipulation and Monitoring.

คณะผู้จัดทำ

นัธววัฒน์	ปริมสิริคุณาวุฒิ	67070501027
วิศิษฐ์	สุวรรณเนาวิ	67070501042
พลวิษฐ์	วัฒนเหมรัตน์	67070501067

เสนอ

ผศ. ราชวิช สโรชวิกสิต

รายงานนี้เป็นส่วนหนึ่งของรายวิชา

CPE333 ระบบปฏิบัติการ (Operating Systems)

ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

ภาคเรียนที่ 1 ปีการศึกษา 2569

Problem 1: Process Manipulation and Monitoring

1. Learn the important Linux commands for manipulating and monitoring the process or jobs :

- 1.1. Discuss the difference of “top” and “ps” command.
- 1.2. What is the purpose of the “nice” command? How can you nice process?
- 1.3. How to kill the processes and/or jobs? Give example in the report.

1.1. Discuss the difference of “top” and “ps” command.

Ans: **ps** คือการ **snapshot** ครั้งเดียว แต่ **top** เป็นแดชบอร์ดแบบเรียลไทม์ ที่อัปเดตตลอดเวลา
ps -> ps (process status) เป็น **snapshot** ของ process ที่กำลังทำงาน ในขณะที่รันคำสั่ง แล้วก็จบการทำงานทันที เป็นข้อมูลแบบ static ดังนั้นต้องรันซ้ำเพื่อดูข้อมูลล่าสุด
top -> top เป็น **interactive and continuously updates**, แสดงรายการ process โดยเรียงลำดับตามการใช้งาน เช่น CPU, memory พร้อมกับระบุข้อมูลระบบโดยรวม เช่น load average, การใช้ memory, การใช้ CPU ในแต่ละ core

1.2.1. What is the purpose of the “nice” command?

Ans : คำสั่ง **'nice'** ใช้กำหนดหรือปรับค่า niceness ของ process ซึ่ง Linux scheduler ใช้เป็นปัจจัยในการกำหนด scheduling preference โดยค่า niceness อยู่ในช่วง -20 (มี preference สูงกว่า) ถึง 19 (มี preference ต่ำกว่า) และค่าเริ่มต้นคือ 0

1.2.2. How can you nice process ?

Ans : การเพิ่มค่า niceness ทำให้ process มี scheduling preference ต่ำลงเมื่อเทียบกับ process อื่น แต่ไม่ได้กำหนดเปอร์เซ็นต์ CPU time แบบตายตัว ตัวอย่างการใช้งาน:

- เริ่ม process, กำหนดค่า nice : **nice -n 10 ./myprogram**
- เปลี่ยนแปลงค่า nice ของ process ที่กำลังทำงาน : **renice -n 5 -p <PID>**

Note: การลดค่า nice (เพิ่มความสำคัญ) ต้องใช้สิทธิ์ root/sudo

1.3. How to kill the processes and/or jobs? Give example in the report.

Ans: วิธีในการ kill process/job มีดังต่อไปนี้ :

- หา PID: **ps aux | grep myprogram** or **pgrep myprogram**
- Kill ตาม PID: **kill <PID>** (ส่ง SIGTERM, เป็นการขอให้หยุดการทำงาน)
- Force kill: **kill -9 <PID>** (ส่ง SIGKILL, เป็นการสั่ง ซึ่งไม่สามารถปฏิเสธได้)
- Kill ตาม name: **killall myprogram** or **pkill myprogram**
- Kill a background job ตาม job number: **kill %1** (โดย 1 คือหมายเลขตามคำสั่ง jobs)

Problem 2: Foreground and Background Jobs

2. Discuss these situations

First situation

- 1) Create a shell script (just a text file), named ss1_1.sh
- 2) In the script, the “sleep” command is called with 10 second.
- 3) Run the script ss1_1.sh (./ss1_1.sh), wait for 10 second.
- 4) Run the script ss1_1.sh again with the sigh “&” in the end of command (./ss1_1.sh &)
- 5) Discuss the difference of both running methods. (3 and 4)

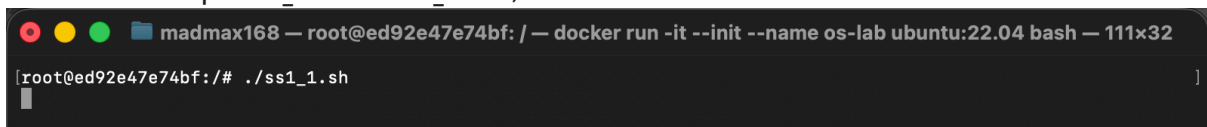
NOTE: If your text file is not runnable, use ‘chmod’ command to fix the issue.
You could try to use a for loop in script and sleep one second per iteration.

- 1) Create a shell script (just a text file), named ss1_1.sh
- 2) In the script, the “sleep” command is called with 10 second.



รูปที่ 1: หลังจากใช้คำสั่ง `nano ss1_1.sh` และเขียน `sleep command 10 sec` ใน script

- 3) Run the script ss1_1.sh (./ss1_1.sh), wait for 10 second.



รูปที่ 2: ผลการรัน script `ss1_1.sh` แบบ foreground เป็นเวลา 10 วินาที

4) Run the script ss1_1.sh again with the sigh “&” in the end of command (./ss1_1.sh &)

```
madmax168 — root@ed92e47e74bf: / — docker run -it --init --name os-lab ubuntu:22.04 bash — 111x32
[root@ed92e47e74bf:/# ./ss1_1.sh
[root@ed92e47e74bf:/# ./ss1_1.sh &
[1] 1298
root@ed92e47e74bf:/#
```

รูปที่ 3: ผลการรัน script ss1_1.sh แบบ background พร้อม job number และ PID

5) Discuss the difference of both running methods. (3 and 4)

Discuss : เมื่อรันแบบ foreground (./ss1_1.sh) shell จะรอให้ foreground job ทำงานเสร็จก่อน จึงยังไม่คืน command prompt ให้ผู้ใช้ป้อนคำสั่งใหม่ใน shell เดิม ส่วนการรันแบบ background (./ss1_1.sh &) shell จะคืน prompt ทันที พร้อมแสดง job number และ PID ทำให้ผู้ใช้ป้อนคำสั่งถัดไปได้ขณะที่ job ยังทำงานอยู่

Second situation

- 1) Create a shell script, named ss1_2.sh
- 2) In the script, the “sleep” command is called with 1000 second.
- 3) Run the script ss1_2.sh (./ss1_2.sh), then press CTRL+Z
- 4) Use “jobs” command, see the result and discuss the result of pressing CTRL+Z
- 5) Show how to use “fg” and “bg” commands to resume the script ss1_2.sh and discuss the difference of both resume commands.

1) Create a shell script, named ss1_2.sh

```
madmax168 — root@ed92e47e74bf: / — docker run -it --init --name os-lab ubuntu:22.04 bash — 111x32
[root@ed92e47e74bf:/# nano ss1_2.sh
root@ed92e47e74bf:/#
```

รูปที่ 4: การสร้าง shell script ชื่อ ss1_2.sh

2) In the script, the “sleep” command is called with 1000 second.

```
madmax168 — root@ed92e47e74bf: / — docker run -it --init --name os-lab ubuntu:22.04 bash — 111x32
GNU nano 6.2 ss1_2.sh *
#!/bin/bash
sleep 1000
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute  ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify  ^_ Go To Line M-E Redo
```

รูปที่ 5: การเขียน sleep command 1000 sec ใน script

3) Run the script ss1_2.sh (./ss1_2.sh), then press CTRL+Z

```
madmax168 — root@ed92e47e74bf: / — docker run -it --init --name os-lab ubuntu:22.04 bash — 111x32
[1]+  Stopped                  ./ss1_2.sh
root@ed92e47e74bf: /#
```

รูปที่ 6: การรัน ss1_2.sh และใช้ Ctrl+Z เพื่อหยุดชั่วคราว

4) Use “jobs” command, see the result and discuss the result of pressing CTRL+Z

```
madmax168 — root@ed92e47e74bf: / — docker run -it --init --name os-lab ubuntu:22.04 bash — 111x32
[1]+  Stopped                  ./ss1_2.sh
root@ed92e47e74bf: /# jobs
[1]+  Stopped                  ./ss1_2.sh
root@ed92e47e74bf: /#
```

รูปที่ 7: การใช้ jobs เพื่อตรวจสอบสถานะของ script

Discuss : การใช้ CTRL + Z เพื่อหยุดการทำงานเพียงชั่วคราวเท่านั้นยังไม่ได้ KILL ตัวนั้น

5) Show how to use “fg” and “bg” commands to resume the script ss1_2.sh and discuss the difference of both resume commands.

```
madmax168 — root@ed92e47e74bf: / — docker run -it --init --name os-lab ubuntu:22.04 bash — 111x32
[1]+  Stopped                  ./ss1_2.sh
root@ed92e47e74bf: /# jobs
[1]+  Stopped                  ./ss1_2.sh
root@ed92e47e74bf: /# fg
./ss1_2.sh
```

รูปที่ 8: การใช้คำสั่ง fg (foreground)

```
root@ed92e47e74bf: /# bg
[2]+ ./ss1_2.sh &
root@ed92e47e74bf: /# jobs
[1]+  Stopped                  ./ss1_2.sh
[2]-  Running                  ./ss1_2.sh &
root@ed92e47e74bf: /#
```

รูปที่ 9: การใช้คำสั่ง bg (background)

Discuss : fg ทำให้การทำงานที่ถูกหยุดไว้ (CTRL + Z) กลับมาทำงานต่อในแบบ foreground ในส่วนของ bg จำนำ job ที่ถูกหยุดการทำงานไว้มารันต่อในรูปแบบ background ดังนั้นสรุปว่า fg หรือ bg นั้นมีไว้เพื่อสลับการทำงานของ job ที่ถูกหยุดการทำงานไว้ ว่าจะทำเป็น foreground หรือ background ไม่ได้เริ่มใหม่

Problem 3: GUI Process Monitoring on Linux

3. Find and try out process monitoring tools with GUI on Linux system

- 3.1. What is it?
- 3.2. How to acquire it?
- 3.3. How to use it?
- 3.4. What value it can monitor?

3.1 What is it?

Ans : เป็น GUI ที่จะแสดงภาพรวมของ process ที่กำลังทำงานทั้ง การใช้ CPU, memory, network, disk คล้ายกับ Task Manager

3.2 How to acquire it?

Ans : โดยปกติ GNOME System Monitor ติดตั้งมาพร้อม Ubuntu อยู่แล้ว หากไม่มี สามารถติดตั้งด้วยคำสั่ง `sudo apt install gnome-system-monitor`

3.3 How to use it?

Ans : ในการใช้งาน เราสามารถเปิดได้ 2 แบบ

- ผ่านเมนู Applications → ค้นหา "System Monitor" แล้วคลิกเปิด
- หรือรันคำสั่งในเทอร์มินัล: `gnome-system-monitor`

หลังจากเปิดขึ้นมา หน้าต่างจะแบ่งเป็น 3 Tabs หลัก:

- **Processes:** ตารางรายชื่อโปรเซสทั้งหมดที่กำลังทำงาน พร้อมคอลัมน์ %CPU, Memory, User ที่รันโปรเซสนั้น
- **Resources:** กราฟแสดงการใช้ CPU/Memory/Network แบบเรียลไทม์
- **File Systems:** แสดงพื้นที่ดิสก์ที่ใช้ไปในแต่ละ partition

วิธีการจัดการ Process : คลิกขวาที่ Process ในแท็บ Processes -> เลือก "End Process" (เทียบเท่า kill) หรือ "Change Priority" (เทียบเท่า nice/renice) ได้โดยตรงจาก GUI โดยไม่ต้องพิมพ์คำสั่ง

3.4 What value it can monitor?

Ans : สามารถดูค่าได้ดังนี้

% การใช้ CPU ต่อโปรเซส และต่อคอร์
การใช้หน่วยความจำ (RAM) และ swap
การอ่าน/เขียนดิสก์ (disk I/O)
การรับส่งข้อมูลผ่านเครือข่าย (network throughput)
ลำดับความสำคัญของโปรเซส (nice value / priority)
จำนวน thread และ PID ของแต่ละโปรเซส
User ที่เป็นเจ้าของโปรเซส

Resources Processes File Systems						
Name	ID	CPU	Memory	Disk Read	Disk Write	
gnome-shell	3124	4.3 %	281.8 MB	94.7 KiB/s	—	
gnome-system-monitor	7946	1.0 %	322.3 MB	—	—	
nautilus	6867	0.1 %	288.0 MB	—	—	
ptyxis	4265	0.1 %	258.5 MB	166.7 KiB/s	—	
gnome-session-service	3110	—	1.5 MB	60.0 KiB/s	—	
gjs	3730	—	16.7 MB	—	—	
python3	6964	—	20.4 MB	521.3 KiB/s	—	
glycin-image-rs	9445	—	593.9 kB	—	—	
gvfsd-metadata	3832	—	725.0 kB	—	—	
ptyxis-agent	4292	—	729.1 kB	—	—	
goa-identity-service	3570	—	1.1 MB	—	—	
ibus-x11	4437	—	1.7 MB	—	—	
(sd-pam)	2782	—	2.1 MB	—	—	
pipewire	2806	—	3.8 MB	—	—	
mpiris-proxy	2811	—	241.7 kB	—	—	
wireplumber	2813	—	5.6 MB	—	—	
pipewire	2825	—	618.5 kB	—	—	
pipewire-pulse	2828	—	7.6 MB	—	—	
gdm-wayland-session	2874	—	507.9 kB	—	—	
gnome-session-init-worker	2895	—	958.5 kB	—	—	
xdg-permission-store	2939	—	540.7 kB	—	—	
gcr-ssh-agent	3079	—	700.4 kB	—	—	
gnome-session-ctl	3080	—	385.0 kB	—	—	
ssh-agent	3081	—	1.0 MB	—	—	
gvfsd	3096	—	938.0 kB	—	—	
gvfsd-fuse	3107	—	782.3 kB	—	—	
at-spi-bus-launcher	3219	—	688.1 kB	—	—	

รูปที่ 10: GNOME System Monitor แสดงรายการ Processes และค่า CPU/Memory ของแต่ละโปรเซส



รูปที่ 11: GNOME System Monitor แสดงค่า CPU, Memory, Network และ Disk ของระบบ

Problem 4: Process Monitoring on Windows

4. Find and try out process monitoring tools on Windows system

- 4.1. What is it?
- 4.2. How to acquire it?
- 4.3. How to use it?
- 4.4. What value it can monitor?

4.1 What is it?

Ans : Task Manager เป็นเครื่องมือของ Windows สำหรับจัดการ ดูแล Process ที่กำลังทำงาน หรือใช้งานอยู่

4.2 How to acquire it?

Ans : โดยปกติจะติดตั้งมาแล้วกับ Windows สามารถเปิดได้ผ่านการคลิกขวาที่ taskbar หรือ **shortcut : Ctrl + Shift + Esc**

4.3 How to use it?

Ans : สามารถเปิดได้ผ่านการคลิกขวาที่ taskbar หรือ **shortcut : Ctrl + Shift + Esc**

หน้าต่างจะแบ่งเป็นหลาย Tabs:

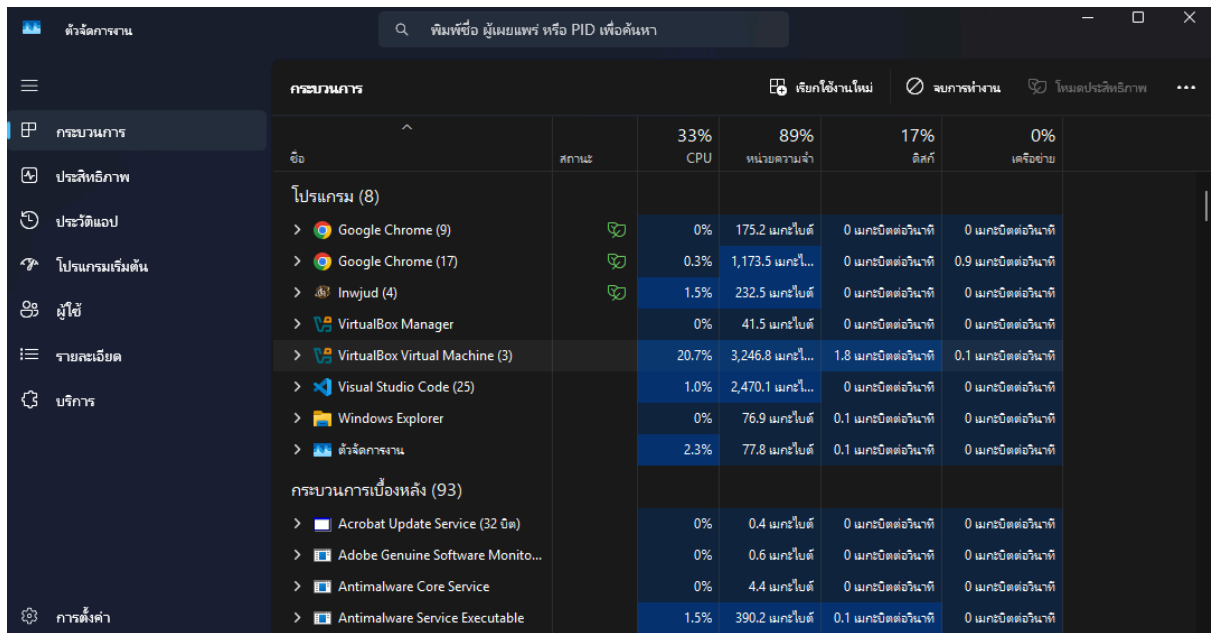
- **Processes:** แสดงแอปและโปรเซสเบื้องหลังทั้งหมด พร้อมคอลัมน์ CPU/Memory/Disk/Network ที่อัปเดตแบบเรียลไทม์ — คลิกที่หัวคอลัมน์เพื่อเรียงลำดับตามการใช้งานสูงสุด
- **Performance:** กราฟภาพรวมของทั้งระบบ (CPU, Memory, Disk, Network, GPU) แบบ historical
- **Startup:** ดูว่าโปรแกรมไหนเปิดพร้อมเครื่องบ้าง และส่งผลกระทบต่อการทำงานของระบบ

การจัดการโปรเซส: ในแท็บ Processes สามารถคลิกขวาเลือก 'End Task' เพื่อยุติโปรเซสได้ ส่วน 'Set Priority' และ 'Set Affinity' ควรดำเนินการผ่านแท็บ Details โดยใช้ 'Set priority' เพื่อปรับ scheduling priority และ 'Set affinity' เพื่อกำหนด logical processors ที่โปรเซสใช้ได้ ถ้าต้องการรายละเอียดลึกกว่านี้ (per-process I/O แบบละเอียด) ให้เปิด Resource Monitor แทน โดยพิมพ์ **resmon** ใน Run dialog (**Win+R**) หรือกดปุ่ม "Open Resource Monitor" จากแท็บ Performance ของ Task Manager

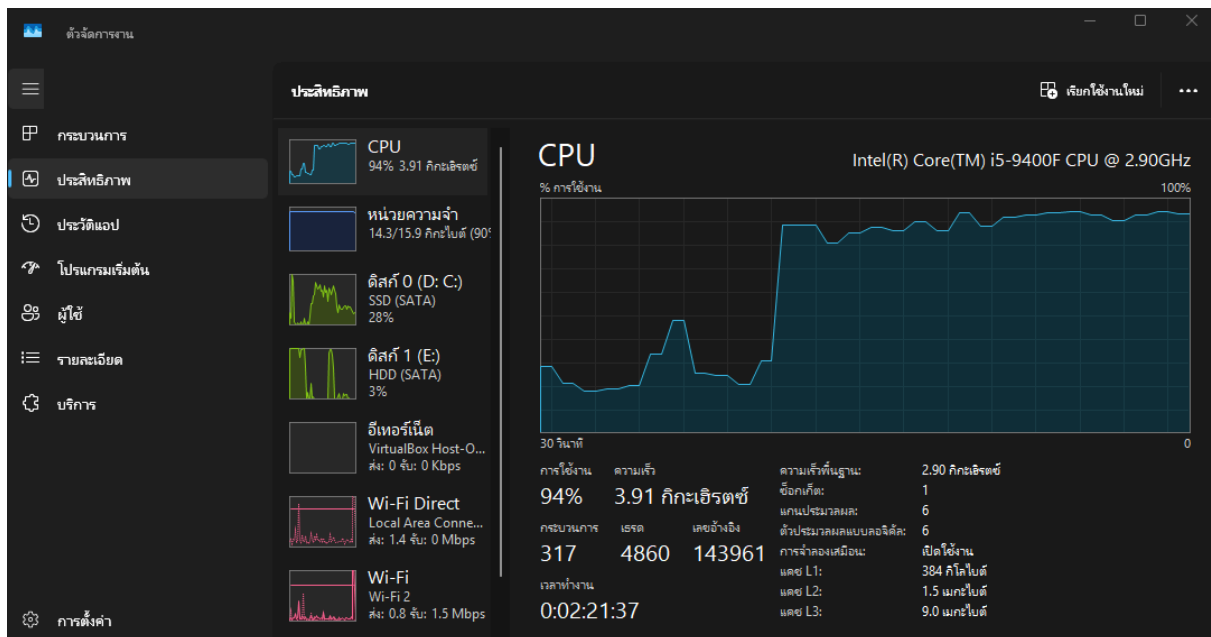
4.4 What value it can monitor?

Ans: สามารถดูค่าได้ดังนี้

- % การใช้ CPU ต่อโปรเซสและต่อ core
- การใช้หน่วยความจำ (Memory) รวมถึง committed memory
- การอ่าน/เขียนดิสก์ (Disk I/O)
- ปริมาณการรับส่งข้อมูลเครือข่าย (Network throughput)
- การใช้ GPU (หากระบบมี GPU ที่ Task Manager รองรับ)
- ผลกระทบต่อเวลาบูตเครื่อง (Startup impact)
- ลำดับความสำคัญของโปรเซส (Priority) และ CPU affinity



รูปที่ 12: Windows Task Manager แสดงรายการ Processes และการใช้ทรัพยากรแบบเรียลไทม์



รูปที่ 13: Windows Task Manager แสดงกราฟการใช้ CPU ในแท็บ Performance

Problem 5: STCF Scheduler Simulation

5. Write C function to simulate the STCF, the program 'PS3.c' that is already provided will read the data in .csv format and convert it into a struct, before sending it to your function, which must process this struct and show which process is running for each time slot.

Assume:

- Your function and context switch occur very fast. You shouldn't count this as CPU usage.
- The process that arrives at time T means "arrives at the very end of time T-1, but before the scheduler updates the ready queue for time T"
- If multiple processes arrive at the same time, sort by using process_id.'
- If multiple processes have the same remaining time, sort by using process_id.'

Note that in the original code, the file 'case1.csv', case2.csv', and 'case3.csv' must be in the same directory as the executable file. And it can be used via the terminal, for example.

./PS3 case1.csv

All of the input cases and the code that processes them are already in LEB2. You may use the PS3.c and fill in the provided blank space in the function with your version of the scheduler.

The expected result that your program needed to print out should look like this:

```
[madmax168@Polwarits-MacBook-Pro ~ % docker cp ~/Downloads/PS3.c os-lab:/root/PS3.c ]
Successfully copied 3.07kB to os-lab:/root/PS3.c
[madmax168@Polwarits-MacBook-Pro ~ % docker cp ~/Downloads/case1.csv os-lab:/root/case1.csv ]
Successfully copied 2.05kB to os-lab:/root/case1.csv
[madmax168@Polwarits-MacBook-Pro ~ % docker cp ~/Downloads/case2.csv os-lab:/root/case2.csv ]
Successfully copied 2.05kB to os-lab:/root/case2.csv
[madmax168@Polwarits-MacBook-Pro ~ % docker cp ~/Downloads/case3.csv os-lab:/root/case3.csv ]
Successfully copied 2.05kB to os-lab:/root/case3.csv
madmax168@Polwarits-MacBook-Pro ~ %
```

รูปที่ 14: การนำไฟล์ PS3.c และ caseX.csv เข้า container

Explain: เนื่องจากผมใช้ cli ผ่าน container ของ docker ดังนั้นการที่จะจัดการ PS3.c ได้ต้องนำไฟล์ดังกล่าวเข้าไปใน container ก่อน

```
[root@ed92e47e74bf:~# nano PS3.c ]
root@ed92e47e74bf:~#
```

รูปที่ 15: การใช้คำสั่ง nano PS3.c เพื่อแก้ไข source code

```
GNU nano 6.2 PS3.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_PROCESSES 100

// you may change this if needed
typedef struct {
    int pid; // process id (int)
    int arrival; // arrival time
    int burst; // burst time
    int remaining; // remaining burst time
    int finished; // 0 = not finished, 1 = done
} Process;

void simulate_stfc(Process *procs, int n) {
    int current_time = 0;
    int finished_count = 0;
    int segment_start = 0;
    int current_running = -2; // -2 not start, -1 = IDLE, >= 0 = PID that running

    while (finished_count < n) {
        int best_index = -1;
        for (int i = 0; i < n; i++) {
            if (procs[i].finished) continue;
            if (procs[i].arrival > current_time) continue;

            if (best_index == -1) {
                best_index = i;
            } else if (procs[i].remaining < procs[best_index].remaining) {
                best_index = i;
            } else if (procs[i].remaining == procs[best_index].remaining &&
                       procs[i].pid < procs[best_index].pid) {
                best_index = i;
            }
        }

        if (best_index == -1) {
            current_time++;
            continue;
        }

        if (current_running != best_index) {
            segment_start = current_time;
        }

        current_running = best_index;
        procs[best_index].remaining--;
        if (procs[best_index].remaining == 0) {
            finished_count++;
        }
        current_time++;
    }
}
```

รูปที่ 16: Source code ภายใน PS3.c และส่วนที่แก้ไขใน simulate_stfc

Explain : ผมได้ทำการแก้ไขโค้ดภายใน PS3.c ด้วยการเพิ่มโค้ดข้างล่างไปใส่ใน simulate_stfc

```
void simulate_stfc(Process *procs, int n) {
    int current_time = 0;
    int finished_count = 0;
    int segment_start = 0;
    int current_running = -2; // -2 = ยังไม่เริ่ม, -1 = IDLE, >=0 = pidที่กำลังรัน

    while (finished_count < n) {
        int best_index = -1;
        for (int i = 0; i < n; i++) {
            if (procs[i].finished) continue;
            if (procs[i].arrival > current_time) continue;

            if (best_index == -1) {
                best_index = i;
            } else if (procs[i].remaining < procs[best_index].remaining) {
                best_index = i;
            } else if (procs[i].remaining == procs[best_index].remaining &&
                       procs[i].pid < procs[best_index].pid) {
                best_index = i;
            }
        }

        if (best_index == -1) {
            current_time++;
            continue;
        }

        if (current_running != best_index) {
            segment_start = current_time;
        }

        current_running = best_index;
        procs[best_index].remaining--;
        if (procs[best_index].remaining == 0) {
            finished_count++;
        }
        current_time++;
    }

    int running_pid = (best_index == -1) ? -1 : procs[best_index].pid;
}
```

```

if (running_pid != current_running) {
    if (current_running != -2) {
        if (current_running == -1) {
            printf("[%d-%d]: IDLE\n", segment_start, current_time);
        } else {
            printf("[%d-%d]: P%d\n", segment_start, current_time, current_running);
        }
    }
    segment_start = current_time;
    current_running = running_pid;
}

if (best_index != -1) {
    procs[best_index].remaining--;
    if (procs[best_index].remaining == 0) {
        procs[best_index].finished = 1;
        finished_count++;
    }
}
current_time++;
}

if (current_running == -1) {
    printf("[%d-%d]: IDLE\n", segment_start, current_time);
} else {
    printf("[%d-%d]: P%d\n", segment_start, current_time, current_running);
}
}

```

Program Overview

STCF (Shortest Time-to-Completion First) หรือที่รู้จักกันในชื่อ preemptive Shortest Job First เป็นอัลกอริทึมที่เลือก process ที่พร้อมทำงาน (ready process) ซึ่งมีเวลาประมวลผลที่เหลืออยู่น้อยที่สุดในการตัดสินใจเลือกงาน (scheduling decision) ในแต่ละครั้ง ตัว scheduler จะตรวจสอบทุก process ที่มาถึงและยังทำงานไม่เสร็จในทุก ๆ ช่วงเวลา หากมีสอง process ที่มีเวลาเหลือเท่ากัน จะเลือก process ที่มี ID น้อยกว่า หากยังไม่มี process ใดมาถึงหรือทำเสร็จหมดแล้ว CPU จะยังคงสถานะ IDLE จนกว่าจะมี process ใหม่มาถึง

Results

- **Case 1:** ผลลัพธ์แสดงให้เห็นว่า process ใหม่ที่มาถึงและมีเวลาเหลือสั้นกว่า จะทำการแย่งชิง (preempt) process ที่กำลังทำงานอยู่ หลังจาก process ที่สั้นกว่าทำเสร็จสิ้น scheduler จะเลือก process ที่มีเวลาเหลือสั้นที่สุดจากบรรดา process ที่พร้อมทำงาน แสดงให้เห็นถึงการทำให้ preempt ที่ถูกต้อง
- **Case 2:** ผลลัพธ์แสดงช่วงเวลา CPU ว่าง (idle interval) ก่อนที่ process ถัดไปจะมาถึง เมื่อมี process พร้อมทำงาน scheduler จะใช้กฎของเวลาที่เหลือ และใช้ process ID เป็นตัวตัดสินเมื่อเวลาที่เหลือเท่ากัน
- **Case 3:** ผลลัพธ์แสดงให้เห็นว่า process ที่มีเวลาเหลือสั้นที่สุดจะถูกเลือกในทุกช่วงเวลา เมื่อ process มีเวลาเหลือเท่ากัน จะเลือก process ที่มี ID ต่ำกว่า ซึ่งเป็นการยืนยันกฎการตัดสินใจในกรณีเสมอ

Discussion

การทำงานนี้เป็น preemptive STCF scheduler เนื่องจากการประเมิน ready queue ใหม่ในทุก ๆ หน่วยเวลา แทนที่จะปล่อยให้ process ที่ถูกเลือกทำงานจนเสร็จสิ้น ทั้ง 3 กรณีแสดงให้เห็นพฤติกรรมที่จำเป็นครบถ้วน ได้แก่ การทำ preempt หลังจากมีงานที่สั้นกว่ามาถึง, ช่วงเวลา CPU ว่างเมื่อไม่มี process พร้อมทำงาน, และการเลือกแบบกำหนดแน่นอนโดยใช้ process ID ที่ต่ำกว่าเมื่อเวลาที่เหลือเท่ากัน โดยยังคงรักษาผลลัพธ์จากการทดลองและตรรกะของ scheduler แบบดั้งเดิมไว้

หลังจากแก้ไข และ save โค้ดใหม่ลง PS3.c แล้วก็จะมาทดสอบ case1, case 2, case 3 ที่เป็นไฟล์ .CSV

```
[root@ed92e47e74bf:~# nano PS3.c ]
[root@ed92e47e74bf:~# gcc -o PS3 PS3.c ]
[root@ed92e47e74bf:~# ./PS3 case1.csv ]
[0-2]: P1
[2-4]: P2
[4-5]: P3
[5-7]: P2
[7-12]: P1
root@ed92e47e74bf:~# █
```

รูปที่ 17: ผลลัพธ์ของ case1.csv

```
[root@ed92e47e74bf:~# ./PS3 case2.csv ]
[0-2]: P1
[2-5]: IDLE
[5-6]: P2
[6-7]: P4
[7-9]: P2
[9-12]: P3
root@ed92e47e74bf:~# █
```

รูปที่ 18: ผลลัพธ์ของ case2.csv

```
[root@ed92e47e74bf:~# ./PS3 case2.csv ]
[0-2]: P1
[2-5]: IDLE
[5-6]: P2
[6-7]: P4
[7-9]: P2
[9-12]: P3
root@ed92e47e74bf:~# █
```

รูปที่ 19: ผลลัพธ์ของ case3.csv